Tanks Battle Royale — Initial Project Proposal

ft_transcendence — Team: hebatist, aneri-da, vfidelis, lbarreto, gada-sil

> **Purpose of this document.** This is a discussion draft, not the final GDD. It lays out roles, working practices, stack options, and a module/points plan so the team can debate and lock decisions together. Anywhere you see **[DECISION NEEDED]**, that's a point the team must agree on before writing the final GDD.

1. What are we doing? (already defined)

A **browser-based battle royale among tanks (2–4 players)**, built with **Phaser**, plus a companion **landing page** for account creation, matchmaking, and rankings.

Already implemented (local, 2 players):

- Tank movement and aiming.
- Firing projectiles that destroy walls (blocks), gasoline barrels, and other tanks.
- Win condition: last tank standing.

To be implemented:

- **Ammo gauge per tank**, shown on-screen (sides): holds 5 shots, each shot depletes it by one. When empty, the tank must wait (~8s) for it to refill before firing again.
- **Shrinking map**: after 3 minutes, the map contracts tile by tile from the outside in (explosion/fire visual), forcing surviving tanks toward the center.
- **Online multiplayer** (replacing local-only play).
- **Landing page**: registration/login, room creation/matchmaking, rankings.

These are product decisions the team has already made — they're restated here so the rest of the document (roles, stack, modules) has a fixed target to plan against.

2. Team Organization & Roles

The subject requires four roles to be explicitly assigned and documented: **Product Owner, Project Manager/Scrum Master, Technical Lead/Architect,** and **Developers** (everyone). With 5 members, roles can stay mostly specialized, with one overlap.

Who fits each role [DECISION NEEDED — team self-selects at kickoff, by fit rather than default]

Rather than pre-assign names, here's what each role actually demands day to day. Read these and self-select — the point is that the person in the seat actually wants what the seat requires, not that a title looks good on the README.

Role	What the job actually is	Traits that make someone good at it
Technical Lead / Architect	Owns the architecture (game loop, netcode, backend split); makes the technology-stack call and defends it; reviews the critical PRs (netcode, DB schema, auth).	Comfortable making irreversible technical bets under uncertainty and owning the consequences. Thinks in systems (data flow, failure modes) rather than just "does it work on my machine." Can explain a trade-off to someone who isn't technical. Says no to scope creep without needing to be liked for it.
Product Owner	Owns the feature backlog and priority order; decides what "done" means for a gameplay feature (gauge, map shrink, matchmaking); is the one who explains "why we built it this way" during evaluation.	Good at making a call and sticking to it rather than re-opening debates. Thinks like a player, not just a builder — notices when a feature is technically correct but feels bad to play. Organized enough to keep a backlog honest. Comfortable being the tie-breaker when two developers disagree.
Project Manager / Scrum Master	Runs the recurring sync, keeps the board current, chases blockers before they become deadline emergencies, tracks who owes what.	Naturally the one who checks in on people without it feeling like nagging. Notices when someone's gone quiet in the channel and follows up early. Detail-oriented, low ego about doing "boring" process work. Diplomatic when a deadline is slipping.
Developers (everyone, including the three above)	Implement features/modules; every PR needs one review before merging to `main`.	Gives and takes code review feedback without ego. Asks for help when stuck for an hour, not for three days. Gives realistic estimates rather than optimistic ones.

Note: with 5 people, one person can (and likely will) hold a leadership role *and* be a developer — the subject expects this. The Tech Lead in particular should stay hands-on in the code, not just review it.

Work-area fit (not an assignment)

The project splits into four surfaces. Below is the *kind of person* each suits — use it as a prompt for who volunteers, not a roster:

Area	Suits someone who...	Covers
Game core (Phaser)	Enjoys "game feel" iteration — tuning numbers, timing, and juiciness through repeated playtesting rather than getting it right the first time.	Gauge system, map-shrink mechanic, game feel
Multiplayer / backend	Likes networking/concurrency problems and is patient debugging things that only fail intermittently (race conditions, desync).	WebSocket sync, matchmaking, room lifecycle
Landing page & user management	Enjoys UI/UX and CRUD-heavy work — cares about forms, validation, and auth flows actually feeling solid.	Auth, profile, friends, rankings UI
DevOps / infra	Is curious about infrastructure tooling and comfortable reading unfamiliar docs independently (Docker, CI).	Docker Compose, CI

3. Recommended Project Management Practices

The subject lists these as *recommended*, not mandatory — but the team should explicitly adopt (or reject) each one so the GDD can state "we do X" rather than leave it implicit.

Practice	Adopted?	How
Regular communication	**Yes**	One weekly sync (fixed day/time, **[DECISION NEEDED]**) + an async Discord channel for daily blockers.
Task organization	**Yes**	GitHub Issues + a GitHub Project board (To Do / In Progress / Review / Done). Avoid a second tool (Trello) to prevent split sources of truth.
Work breakdown	**Yes**	Backlog broken into issues per feature/module, each small enough to land in ≤ 2–3 days.
Code reviews	**Yes**	Branch-per-feature, **at least 1 approval required before merging to `main`**, no direct pushes to `main`.
Documentation	**Yes**	Decisions and "why" go in the README/GDD, not just Discord — Discord history disappears, the repo doesn't.
Communication channel	**Yes**	Discord server (channels: `#general`, `#backend`, `#game`, `#frontend`, `#devops`).
Commit convention	**Yes (added)**	Conventional-ish prefixes (`feat:`, `fix:`, `chore:`) so `git log` stays legible with 5 people committing.
Definition of Done	**Yes (added)**	A feature is "done" when: it works locally, has been reviewed, and has no console errors/warnings (subject requirement).

4. Stack

4.1 Game

Phaser — already decided, already in progress. No debate needed here.

4.2 Frontend (landing page)

[DECISION NEEDED] — Pure React, Angular, or Vue. All three satisfy the subject's "frontend framework" requirement equally (see §5, Web module). Pick based on team familiarity rather than the module rules, since none of them scores differently.

4.3 Backend

Three candidates were shortlisted. None is disqualifying; the choice should weigh **real-time/WebSocket ergonomics** (the game needs low-latency tank-position sync) against **team familiarity** and **delivery speed** for a 5-person team on a fixed deadline.

	ASP.NET Core (C#)	Spring Boot (Java)	Ruby on Rails
Real-time / WebSockets	Strongest fit: **SignalR** gives real-time groups, automatic reconnection, and fallback transports out of the box — exactly the "handle	Solid via `spring-websocket`/STOMP, but more manual wiring than SignalR; reconnection logic is on you.	Weakest fit: **Action Cable** works but isn't built for high-frequency, low-latency tick updates across several rooms — likely needs a separate lightweight WS service

	disconnection/reconnection gracefully" requirement for the Remote Players module.		alongside Rails, i.e. a second backend to maintain.
Dev speed (5-person team)	Medium — less boilerplate than Spring, but the team likely has the least prior C# exposure.	Slower — real ceremony (DTOs, config, layers); mature and well-documented, but heavier to move fast in.	**Fastest for CRUD** — Devise (auth), scaffolding, and convention-over-configuration make the landing page/user-management side very quick to build.
ORM	EF Core — mature, first-class migrations.	Spring Data JPA/Hibernate — very mature, more configuration surface.	ActiveRecord — famously the fastest to get a working schema+API from zero.
Auth / OAuth	ASP.NET Identity — good OAuth2/2FA support out of the box.	Spring Security — excellent OAuth2/JWT support, but more config.	Devise + OmniAuth — quick to wire, huge community precedent.
Container footprint	Small, fast cold start.	Heavier images, slower JVM cold start (matters for the "single-command" deploy requirement, though not blocking).	Small, fast cold start.
Team learning curve	Moderate — new language (C#) for most.	Moderate–high — Java + Spring's conventions/annotations take longer to get comfortable with than the syntax alone suggests.	Low if any member knows Ruby; otherwise moderate — the syntax is easy, but "Rails magic" has its own learning curve.
Fit for this game specifically	Best out-of-the-box fit for room-based real-time multiplayer.	Good if the team leans into the DevOps "microservices" module (Spring's ecosystem is built for that split).	Best if the team wants to prioritize the landing page/ranking/user-management side and treat the game server as a mostly separate concern.

Read this table as: if the WebSocket/netcode quality of the tank battles is the priority, ASP.NET Core (SignalR) has the clearest built-in answer. If the team wants to lean into the DevOps microservices module later, Spring Boot's ecosystem rewards that. If the priority is shipping the landing page (auth, rooms, rankings) fast and treating the game server as a separate, smaller service, Rails gets there quickest but likely means a two-backend architecture.

[DECISION NEEDED] — pick one, and record *why* in the GDD's Technical Stack section (the subject requires justification for major technical choices).

5. Chapter IV — Modules & Points Balance

Target: 14 points minimum (major = 2, minor = 1). The subject explicitly recommends aiming above 14 as a buffer against a module failing validation during evaluation.

Per team direction:

- **Excluded:** Accessibility & Internationalization, Cybersecurity, Blockchain — not pursuing these categories.
- **Priority:** Gaming and user experience, User Management.
- **Maybe:** Artificial Intelligence (AI opponent for solo/practice play).
- **Minimal footprint:** DevOps — containerization is already a *mandatory baseline requirement* (not a scored module), so only a light-touch DevOps module is proposed on top of that.

5.1 "Free" points — dictated by the game we already decided to build

These aren't really choices — they fall out of building *this specific game* (2–4 players, online, real-time). Listing them separately shows how much of the 14 is already earned by product decisions alone.

Category	Module	Type	Pts	Why it's "free"
Gaming & UX	Complete web-based game	Major	2	The tank game itself.
Gaming & UX	Remote players	Major	2	"Online multiplayer will be implemented."
Gaming & UX	Multiplayer 3+ players	Major	2	"Battle royale among tanks (2 to 4)."
Web	Real-time features (WebSockets)	Major	2	Required to sync tank positions/fire events at all.
Web	Frontend + backend framework	Major	2	Required to build the landing page + API regardless of modules.
Subtotal			**10**	

5.2 Two modules to cross the 14-point line

Category	Module	Type	Pts	Notes
User Management	Standard user management & auth	Major	2	Profile, avatar, friends, online status — needed for the landing page anyway.
Artificial Intelligence	AI Opponent	Major	2	Rule-based bot tank (not ML) for solo/practice play; reuses the existing game, no new infra. Covers the team's "maybe."
Running total			**14**	**Minimum reached.**

5.3 Buffer / stretch modules (recommended, matches stated interests)

Beyond the minimum, these follow the team's stated priorities (Gaming, User Management) and build on what's already planned:

Category	Module	Type	Pts	Dependency
Web	User interaction: chat + profile + friends	Major	2	None — can share implementation with the User Management module above.
User Management	Game stats & match history	Minor	1	Requires a game (have one). Feeds the rankings feature already planned.
User Management	Remote auth (OAuth 2.0: Google/GitHub/42)	Minor	1	None.
Gaming & UX	Tournament system	Minor	1	Requires a game (have one).
Gaming & UX	Gamification (achievements/leaderboard/XP)	Minor	1	Requires a game (have one). Pairs naturally with the ammo-gauge/ranking systems.
DevOps	Health check / status page + backups	Minor	1	None. Deliberately the *lightest* DevOps module — ELK, Prometheus/Grafana, and microservices are available but not recommended unless someone specifically wants that scope.
Buffer subtotal			**7**	

5.4 Grand total if all of the above ships

10 (free) + 4 (to reach minimum) + 7 (buffer) = 21 points, against a 14-point requirement — a healthy margin in case any single module doesn't validate during evaluation, without touching Accessibility/i18n, Cybersecurity, or Blockchain.

> ⚠️ **Reality check for the team:** 21 points is a *menu*, not a mandate. §5.1–5.2 (14 pts) is the safe floor. §5.3 is explicitly the first thing to cut if the team is behind schedule — pick 1–2 of those, not all 6, once velocity is known.

Not selected (available if priorities change later)

- **Data & Analytics** — an "Advanced analytics dashboard" (Major, 2 pts) would reuse the same match-history data as §5.3's Game Stats module, if the team wants another buffer point later.
- **Devops heavier options** (ELK log pipeline, Prometheus/Grafana monitoring, microservices split) — available, but sized for a team member specifically interested in infra; not in the recommended plan above.

Blockchain is not in this list — it's an **excluded** category (see above), not a deprioritized one. The team decided against it outright, rather than leaving it as a future option.

6. Open decisions before writing the final GDD

1. Confirm/reassign the four roles in §2.
2. Pick the frontend framework (React / Angular / Vue) — §4.2.
3. Pick the backend framework (ASP.NET Core / Spring Boot / Rails) — §4.3.
4. Confirm the weekly sync day/time — §3.
5. Decide how many §5.3 buffer modules to actually commit to (recommend: 2–3, not all 6).